



ANGELES UNIVERSITY FOUNDATION
COLLEGE OF COMPUTER STUDIES
ANGELES CITY

PDC50

PROFESSIONAL DOMAIN COURSE 5

LAB QUIZ REACT

SHOPSMART CATALOG AND CART CHALLENGE

Prepared By:

Stephanie Joy Revano

John Kerby Sambile

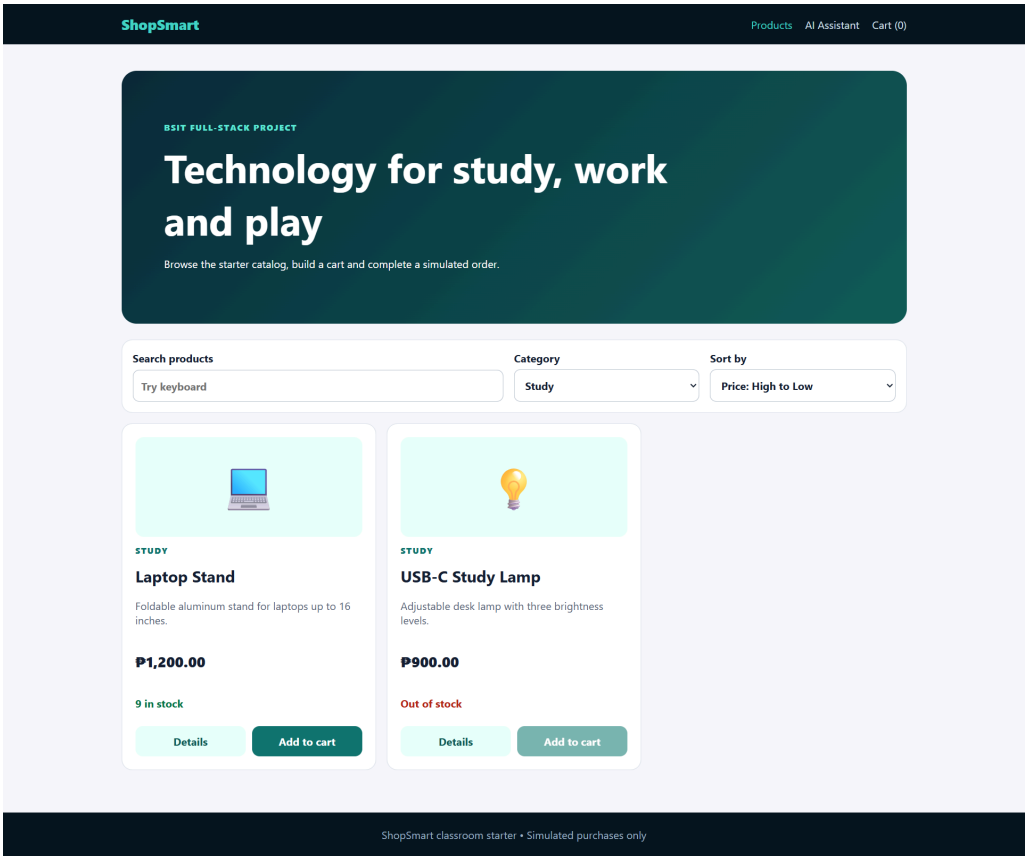
Project / repository

Field	Entry
COGNIX--ShopSmart-Challenge	https://github.com/teptepphhh/COGNIX--ShopSmart-Challenge.git

Activity 1: Sort the Product Catalog

Goal: a labeled Sort control in the product toolbar with Default, Price low to high, Price high to low, and Name A to Z, working together with search and category.

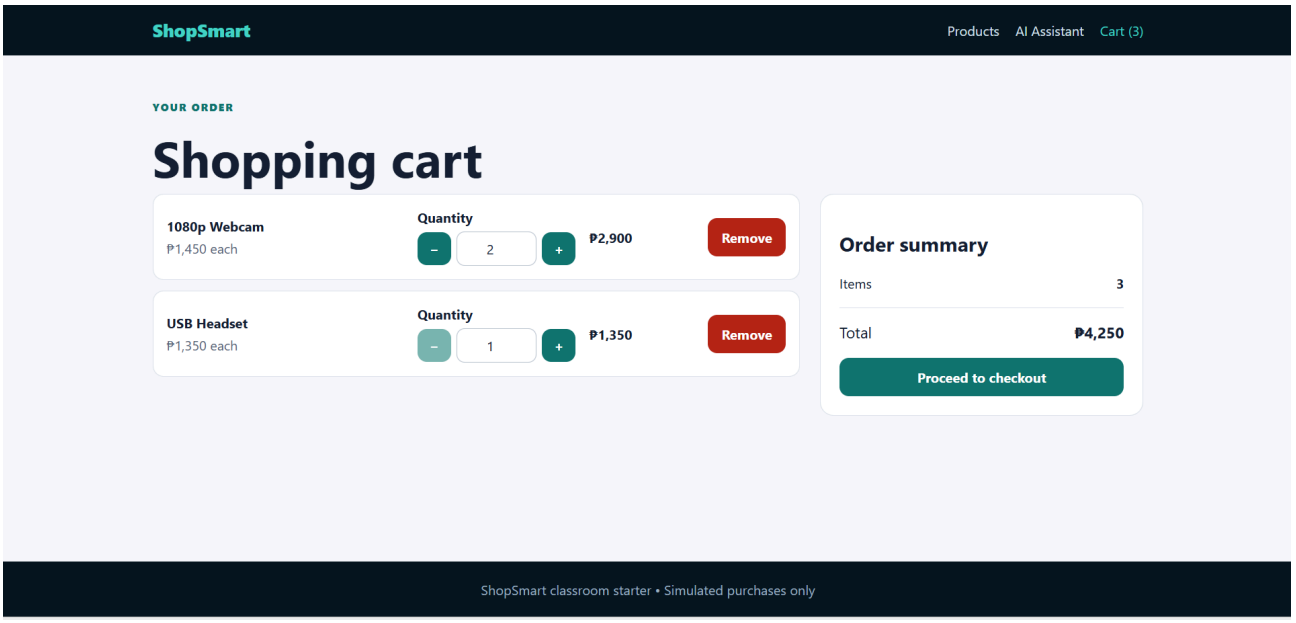
Screenshot 1: Filtered and sorted catalog



Activity 2: Make Cart Quantities Easier to Change

Goal: a minus button and a plus button for each cart item that use the existing cart update behavior, while the existing number input keeps working.

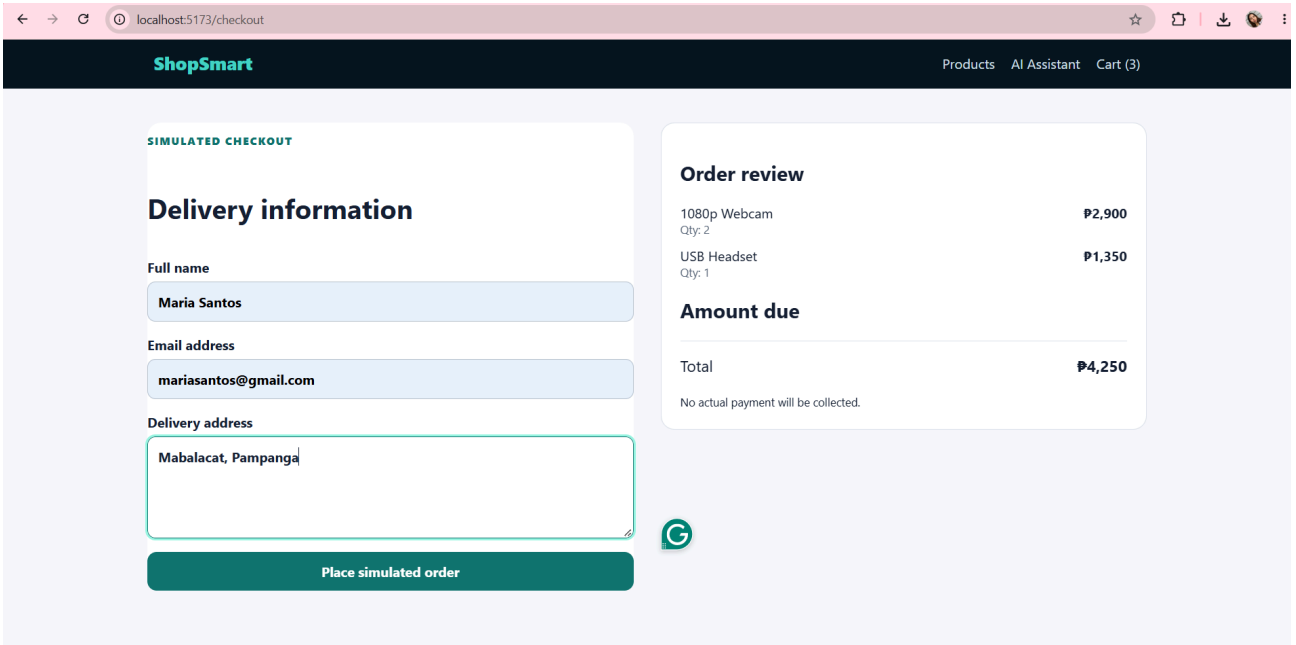
Screenshot 2: Cart with two products and changed quantity

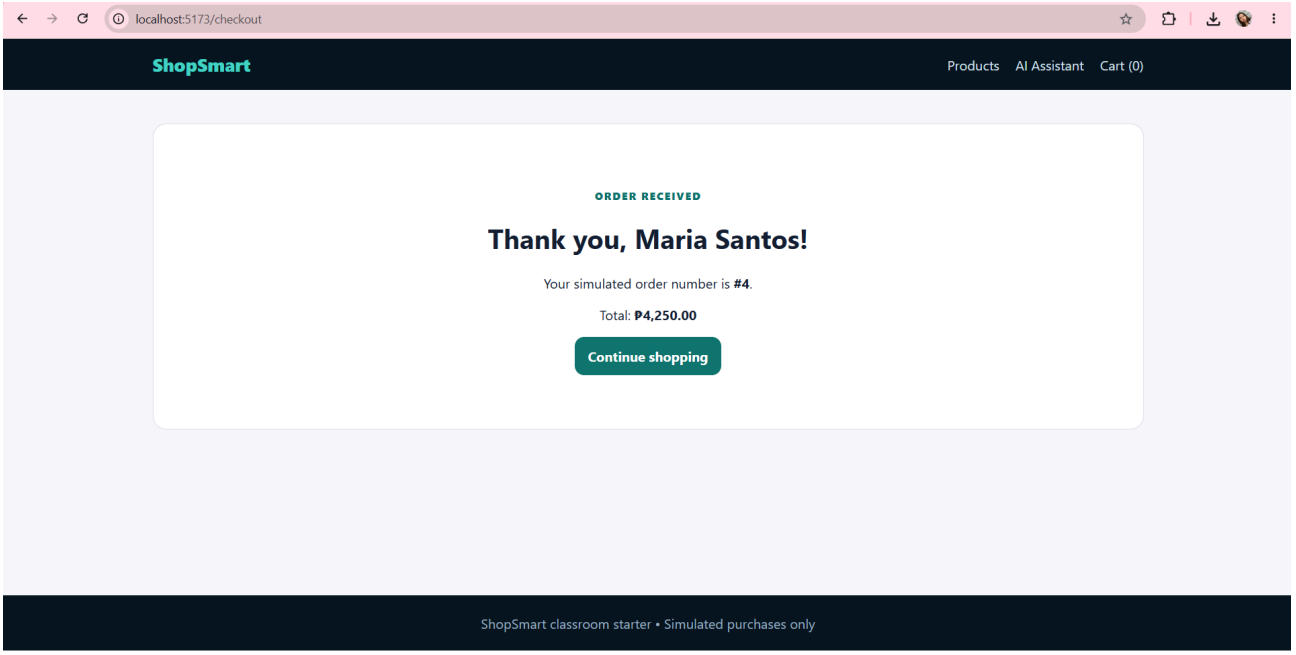


Activity 3: Show an Itemized Checkout Review

Goal: the checkout summary shows each product name, its quantity, and its line subtotal, while keeping the existing total and form.

Screenshot 3: Checkout review with line items and total





Final npm run check Result

Run npm run check from the ShopSmart-Starter root. It runs linting, tests, and a frontend build. The submission must show the final run completing successfully.

```
PS C:\Users\Stephanie Joy Revano\Documents\BSIT4A-PDC50\ShopSmart-Starter> npm run check

> shopsmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build


> shopsmart-frontend@1.0.0 lint
> eslint .


> shopsmart-backend@1.0.0 lint
> eslint .

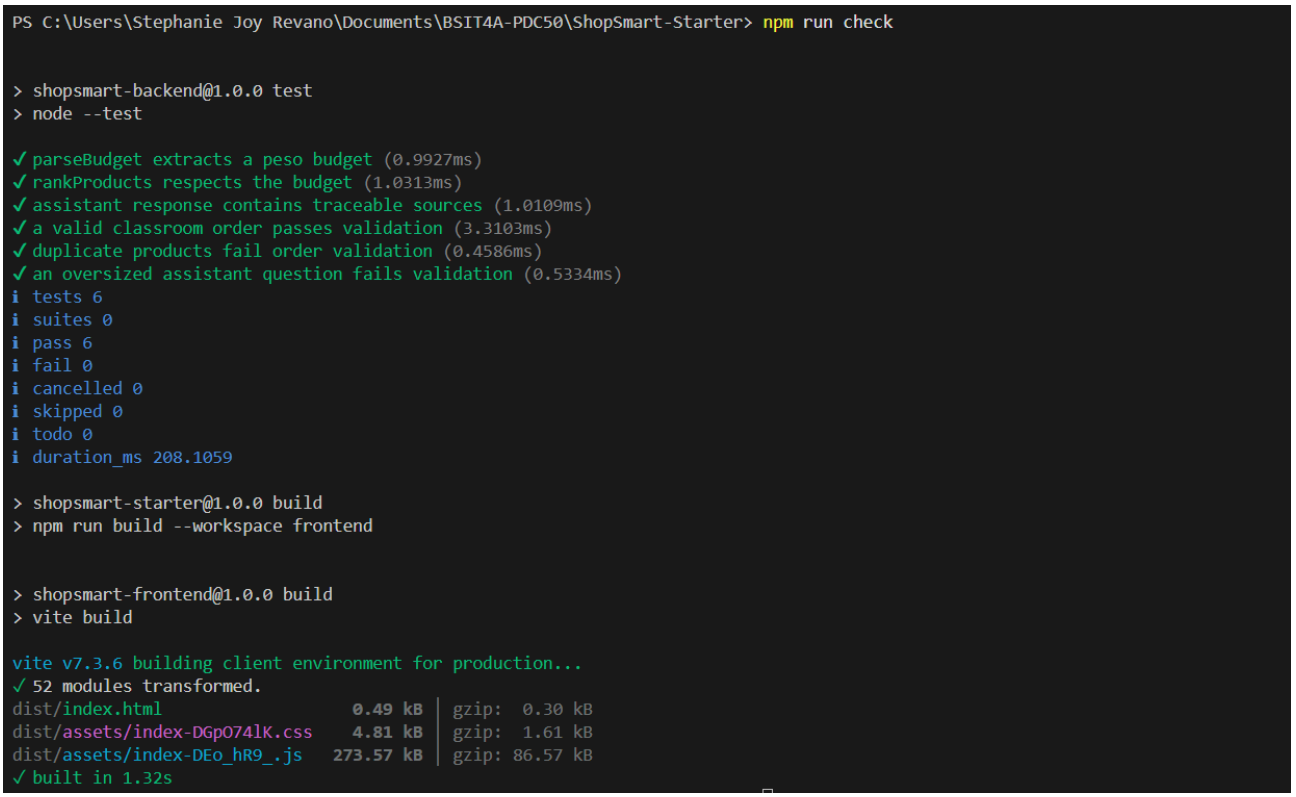

> shopsmart-starter@1.0.0 test
> npm run test --workspace frontend && npm run test --workspace backend


> shopsmart-frontend@1.0.0 test
> vitest run


 RUN  v3.2.7 C:/Users/Stephanie Joy Revano/Documents/BSIT4A-PDC50/ShopSmart-Starter/frontend

 ✓ src/utils/cart.test.js (4 tests) 4ms
   ✓ cart utilities > counts all cart units and handles an empty cart 2ms
   ✓ cart utilities > calculates the cart total 0ms
   ✓ cart utilities > adds a new product 0ms
   ✓ cart utilities > prevents a quantity above stock 0ms

 Test Files  1 passed (1)
      Tests  4 passed (4)
   Start at  17:20:46
```



Written Explanation: where the sort choice is stored, how sorting and filtering work together, why the quantity buttons reuse the cart update rule, and how the checkout review gets its values.

In ProductsPage.jsx the chosen sorting option is kept in the sort state and by default the products remain in their original order. When a customer carries out a search or picks a category, the catalog is first filtered using the current filters before the selected sorting option is applied to the results. The sorting is carried out on a copy of the product list after it has been filtered, thus the original product data and its default order remain unaltered. With regard to the cart, the new plus and minus buttons make use of the existing setQuantity function rather than introducing a separate quantity rule. This is so that the quantity logic remains in one place since setQuantity already makes use of updateCartQuantity to enforce limits, for example by preventing the quantity from going below 1 or above the available stock. The checkout review obtains its information directly from the current cart data supplied by useCart(), which means that the items and quantities displayed stay in line with those actually in the customer's cart. Each item displays its product name, quantity and line subtotal using the existing cart data and the calculateItemSubtotal() function. The total amount that is currently in the cart is then shown as the overall sum the customer has to pay.